

# 计算几何

Morning Flower and Evening Oath

XCPC vector 模板分册

# 目录

|                              |           |
|------------------------------|-----------|
| 使用约定                         | ii        |
| <b>第一章 计算几何</b>              | <b>1</b>  |
| 1.1 整数精确二维几何                 | 1         |
| 1.2 浮点点线圆构造                  | 3         |
| <b>第二章 精确有界半平面交</b>          | <b>7</b>  |
| 2.1 整数半平面的有界多边形交             | 7         |
| <b>第三章 圆与多边形面积</b>           | <b>10</b> |
| 3.1 圆与简单多边形相交面积              | 10        |
| <b>第四章 最小覆盖圆</b>             | <b>12</b> |
| 4.1 随机增量最小覆盖圆                | 12        |
| <b>第五章 圆的切线</b>              | <b>14</b> |
| 5.1 两圆公切线与过点切线               | 14        |
| <b>第六章 浮点最近点对</b>            | <b>16</b> |
| 6.1 浮点平面最近点对                 | 16        |
| <b>第七章 精确几何进阶</b>            | <b>18</b> |
| 7.1 整数最近点对与 Minkowski 和      | 18        |
| 7.2 整数三维共线、共面与方向             | 21        |
| <b>第八章 凸包支撑点查询</b>           | <b>22</b> |
| 8.1 支撑点查询恢复下凸包               | 22        |
| <b>第九章 配套知识与验证范围</b>         | <b>24</b> |
| 9.1 数值算法与几何                  | 24        |
| <b>第十章 计算几何：精确判定与适用范围</b>    | <b>25</b> |
| 10.1 整数系数的有界半平面交             | 25        |
| 10.2 最近点对、Minkowski 和与三维整数判定 | 26        |
| 10.3 浮点最近点对                  | 27        |
| 10.4 公切线与过点切线                | 28        |
| 10.5 随机增量最小覆盖圆               | 29        |
| 10.6 圆与简单多边形相交面积             | 30        |

# 使用约定

C++20，默认包含 `bits/stdc++.h` 并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册按“大类（分册）—种类（章）—项目（节）”组织。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

# 第一章 计算几何

## 1.1 整数精确二维几何

坐标绝对值  $\leq 10^{12}$ ，点数  $\leq 10^6$ 。叉积、点积、线段相交、凸包、点在多边形、点在凸包、旋转卡壳直径。凸包逆时针，不保留共线内点，不重复首点。contains 返回 0 外部、1 边界、2 内部。convex\_contains/diameter2 输入必须为本模板 strict hull。PolarLess 要求排除零向量。

```
1 struct IntegerGeometry
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
7     struct Point
8     {
9         ll x = 0, y = 0;
10
11         bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }
12
13         bool operator==(Point b) const { return x == b.x && y == b.y; }
14     };
15
16     static I cross(Point a, Point b, Point c)
17     {
18         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
19     }
20
21     static I dot(Point a, Point b, Point c)
22     {
23         return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
24     }
25
26     static I dist2(Point a, Point b) { return dot(a, b, b); }
27
28     static int sign(I x) { return (x > 0) - (x < 0); }
29
30     static bool on_segment(Point p, Point a, Point b)
31     {
32         return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
33     }
34
35     static bool intersect(Point a, Point b, Point c, Point d)
36     {
37         int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
38         int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
39         if (!x && on_segment(c, a, b)) return true;
40         if (!y && on_segment(d, a, b)) return true;
41         if (!z && on_segment(a, c, d)) return true;
42         if (!w && on_segment(b, c, d)) return true;
43         return x * y < 0 && z * w < 0;
44     }
```

```

45
46 static vector<Point> hull(vector<Point> p)
47 {
48     sort(p.begin(), p.end());
49     p.erase(unique(p.begin(), p.end()), p.end());
50     int n = (int)p.size();
51     if (n < 3) return p;
52     vector<Point> h;
53     for (Point v : p)
54     {
55         while (h.size() > 1 && cross(h[h.size() - 2], h.back(), v) <= 0)
56             h.pop_back();
57         h.push_back(v);
58     }
59     int low = (int)h.size();
60     for (int i = n - 2; i >= 0; i--)
61     {
62         while ((int)h.size() > low && cross(h[h.size() - 2], h.back(), p[i]) <= 0)
63             h.pop_back();
64         h.push_back(p[i]);
65     }
66     h.pop_back();
67     return h; // CCW, no repeated endpoint, no collinear interior points.
68 }
69
70 static I area2(const vector<Point> &p)
71 {
72     I ans = 0;
73     int n = (int)p.size();
74     for (int i = 0; i < n; i++)
75         ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
76     return ans;
77 }
78
79 // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
80 static int contains(const vector<Point> &p, Point q)
81 {
82     int winding = 0, n = (int)p.size();
83     for (int i = 0; i < n; i++)
84     {
85         Point a = p[i], b = p[(i + 1) % n];
86         if (on_segment(q, a, b)) return 1;
87         I c = cross(a, b, q);
88         if (a.y <= q.y && b.y > q.y && c > 0) ++winding;
89         if (a.y > q.y && b.y <= q.y && c < 0) --winding;
90     }
91     return winding ? 2 : 0;
92 }
93
94 // Input must be the strict CCW hull above. O(log n).
95 static int convex_contains(const vector<Point> &p, Point q)
96 {
97     int n = (int)p.size();
98     if (n < 3) return contains(p, q);
99     I a = cross(p[0], p[1], q), b = cross(p[0], p[n - 1], q);
100     if (a < 0 || b > 0) return 0;

```

```

101     if (!a) return on_segment(q, p[0], p[1]) ? 1 : 0;
102     if (!b) return on_segment(q, p[0], p[n - 1]) ? 1 : 0;
103     int l = 1, r = n - 1;
104     while (l + 1 < r)
105     {
106         int m = (l + r) / 2;
107         if (cross(p[0], p[m], q) >= 0)
108             l = m;
109         else
110             r = m;
111     }
112     I c = cross(p[l], p[r], q);
113     return c < 0 ? 0 : c == 0 ? 1 : 2;
114 }
115
116 static I diameter2(const vector<Point> &p)
117 {
118     int n = (int)p.size();
119     if (n < 2) return 0;
120     if (n == 2) return dist2(p[0], p[1]);
121     I ans = 0;
122     int j = 1;
123     for (int i = 0; i < n; i++)
124     {
125         int k = (i + 1) % n;
126         while (cross(p[i], p[k], p[(j + 1) % n]) > cross(p[i], p[k], p[j]))
127             j = (j + 1) % n;
128         ans = max({ans, dist2(p[i], p[j]), dist2(p[k], p[j])});
129     }
130     return ans;
131 }
132
133 struct PolarLess
134 {
135     static int half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }
136
137     bool operator()(Point a, Point b) const
138     {
139         if (half(a) != half(b)) return half(a) < half(b);
140         I c = cross({0, 0}, a, b);
141         if (c) return c > 0;
142         return dist2({0, 0}, a) < dist2({0, 0}, b);
143     }
144 };
145 };

```

## 1.2 浮点点线圆构造

long double, 显式 eps 默认  $10^{-12}$ 。要求有限输入及适当尺度；接近相切时按容差分类，不能提供任意实数输入的精确拓扑保证。Result 区分 none/one/two/infinite/degenerate。圆圆交点仅在圆心和正半径完全相同时返回 infinite；零半径圆按点处理，不把近同心圆当作重合。圆相关交点容差按局部距离和半径缩放，不使用固定的单位长度下限。零长度线返回 degenerate；投影与点到线段距离将零线段视为点。overlap 返回交叠面积，小圆弓用级数避免三角函数相减消失。坐标缩放和误差预算仍须按题面复核。

```

1 struct RealGeometry
2 {
3     using R = long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6
7     RealGeometry(R eps = 1e-12L) : eps(eps) { assert(eps > 0); }
8
9     struct Point
10    {
11        R x = 0, y = 0;
12
13        Point operator+(Point b) const { return {x + b.x, y + b.y}; }
14
15        Point operator-(Point b) const { return {x - b.x, y - b.y}; }
16
17        Point operator*(R k) const { return {x * k, y * k}; }
18
19        Point operator/(R k) const { return {x / k, y / k}; }
20    };
21
22    struct Circle
23    {
24        Point o;
25        R r;
26    };
27    enum class Kind
28    {
29        none,
30        one,
31        two,
32        infinite,
33        degenerate
34    };
35
36    struct Result
37    {
38        Kind kind;
39        vector<Point> p;
40    };
41
42    static R dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
43
44    static R cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
45
46    static R norm(Point a) { return hypotl(a.x, a.y); }
47
48    static Point perp(Point a) { return {-a.y, a.x}; }
49
50    Point projection(Point p, Point a, Point b) const
51    {
52        Point v = b - a;
53        R d = dot(v, v);
54        if (d == 0) return a;
55        return a + v * (dot(p - a, v) / d);
56    }

```

```

57
58 R segment_distance(Point p, Point a, Point b) const
59 {
60     Point v = b - a;
61     R d = dot(v, v);
62     if (d == 0) return norm(p - a);
63     R t = clamp(dot(p - a, v) / d, R(0), R(1));
64     return norm(p - (a + v * t));
65 }
66
67 Result line_intersection(Point a, Point b, Point c, Point d) const
68 {
69     Point u = b - a, v = d - c;
70     R nu = norm(u), nv = norm(v);
71     if (nu == 0 || nv == 0) return {Kind::degenerate, {}};
72     R det = cross(u, v);
73     if (fabsl(det) <= eps * nu * nv)
74         return {fabsl(cross(c - a, u)) <= eps * nu * max(R(1), norm(c - a))
75                 ? Kind::infinite
76                 : Kind::none,
77                 {}};
78     return {Kind::one, {a + u * (cross(c - a, v) / det)}};
79 }
80
81 Result line_circle(Point a, Point b, Circle c) const
82 {
83     assert(c.r >= 0);
84     Point v = b - a;
85     R len = norm(v);
86     if (len == 0) return {Kind::degenerate, {}};
87     Point h = projection(c.o, a, b);
88     R d = norm(h - c.o), tol = eps * max(d, c.r);
89     if (d > c.r + tol) return {Kind::none, {}};
90     if (fabsl(d - c.r) <= tol) return {Kind::one, {h}};
91     R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
92     Point w = v * (t / len);
93     return {Kind::two, {h - w, h + w}};
94 }
95
96 Result circles(Circle a, Circle b) const
97 {
98     assert(a.r >= 0 && b.r >= 0);
99     Point v = b.o - a.o;
100    R d = norm(v);
101    R tol = eps * max({d, a.r, b.r});
102    // Infinite intersections require identical positive-radius circles.
103    if (d == 0)
104    {
105        if (a.r != b.r) return {Kind::none, {}};
106        if (a.r == 0) return {Kind::one, {a.o}};
107        return {Kind::infinite, {}};
108    }
109    if (a.r == 0 && b.r == 0) return {Kind::none, {}};
110    if (a.r == 0 || b.r == 0)
111    {
112        R radius = max(a.r, b.r);

```

```

113         if (fabsl(d - radius) > tol) return {Kind::none, {}};
114         return {Kind::one, {a.r == 0 ? a.o : b.o}};
115     }
116     if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol) return {Kind::none, {}};
117     R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
118     Point h = a.o + v * (x / d);
119     // Near-concentric circles must not be mistaken for internal tangency.
120     if (fabsl(x) > a.r + tol) return {Kind::none, {}};
121     if (fabsl(fabsl(x) - a.r) <= tol) return {Kind::one, {h}};
122     R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
123     Point w = perp(v) * (y / d);
124     return {Kind::two, {h - w, h + w}};
125 }
126
127 R overlap(Circle a, Circle b) const
128 {
129     assert(a.r >= 0 && b.r >= 0);
130     R d = norm(a.o - b.o), pi = acosl(-1.L);
131     if (d >= a.r + b.r) return 0;
132     if (d <= fabsl(a.r - b.r)) return pi * min(a.r, b.r) * min(a.r, b.r);
133     R difference = (a.r - b.r) * (a.r + b.r);
134     R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
135     R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
136     auto segment = [](R angle)
137     {
138         if (angle < 1e-3L)
139         {
140             R z = angle * angle;
141             return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
142         }
143         return angle - sinl(2 * angle) / 2;
144     };
145     return a.r * a.r * segment(x) + b.r * b.r * segment(y);
146 }
147 };

```

## 第二章 精确有界半平面交

### 2.1 整数半平面的有界多边形交

`bounded_polygon(lines)` 求  $a*x+b*y \leq c$  的交集闭包顶点,  $\text{Line}=\{a,b,c\}$ , 三个整数系数绝对值均  $\leq 10^9$ 。要求交集有界, 空集也允许; 只返回正面积多边形, 点或线段交集返回空向量。无界输入不在接口范围内。 $\text{Point}=\{x,y,d\}$  表示  $(x/d,y/d)$ ,  $d>0$ , 分数不必约分; 按逆时针返回, 不重复首点, 相邻重复点已去除。排序与所有构造、判定使用整数和 `int128`, 不使用 `epsilon`。 $O(n \log n)$  时间、 $O(n)$  空间。浮点转换与面积计算需单独处理。

```
1 struct IntegerHalfplanes
2 {
3     using I = __int128_t;
4     using ll = long long;
5
6     struct Line
7     {
8         ll a, b, c;
9     };
10
11     struct Point
12     {
13         ll x, y, d;
14     };
15
16     static I cross(Line u, Line v) { return I(u.a) * v.b - I(u.b) * v.a; }
17
18     static int half(Line u) { return u.b < 0 || (u.b == 0 && u.a < 0); }
19
20     static Point meet(Line u, Line v)
21     {
22         I d = cross(u, v);
23         assert(d != 0);
24         I x = I(u.c) * v.b - I(u.b) * v.c;
25         I y = I(u.a) * v.c - I(u.c) * v.a;
26         if (d < 0)
27         {
28             d = -d;
29             x = -x;
30             y = -y;
31         }
32         return {(ll)x, (ll)y, (ll)d};
33     }
```

接上页同一结构体与函数：

```

35     static I value(Line u, Point p)
36     {
37         return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
38     }
39
40     static bool same(Point p, Point q)
41     {
42         return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
43     }
44
45     static vector<Point> bounded_polygon(vector<Line> lines)
46     {
47         vector<Line> h;
48         for (auto u : lines)
49         {
50             assert(-10000000000 <= u.a && u.a <= 10000000000);
51             assert(-10000000000 <= u.b && u.b <= 10000000000);
52             assert(-10000000000 <= u.c && u.c <= 10000000000);
53             if (!u.a && !u.b)
54             {
55                 if (u.c < 0) return {};
56             }
57             else
58                 h.push_back(u);
59         }
60         sort(h.begin(),
61              h.end(),
62              [](Line u, Line v)
63              {
64                  if (half(u) != half(v)) return half(u) < half(v);
65                  I d = cross(u, v);
66                  if (d) return d > 0;
67                  ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
68                  ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
69                  return I(u.c) * y < I(v.c) * x;
70              });

```

接上页同一结构体与函数：

```

71     deque<Line> q;
72     for (int i = 0; i < (int)h.size(); i++)
73     {
74         Line u = h[i];
75         if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0) continue;
76         while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
77             q.pop_back();
78         while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0) q.pop_front();
79         if (!q.empty() && cross(q.back(), u) == 0) return {};
80         q.push_back(u);
81     }
82     while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
83         q.pop_back();
84     while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0) q.pop_front();
85     if (q.size() < 3 || cross(q.front(), q.back()) == 0) return {};
86     vector<Point> answer;
87     for (int i = 0; i < (int)q.size(); i++)
88     {
89         Point p = meet(q[i], q[(i + 1) % q.size()]);
90         if (answer.empty() || !same(answer.back(), p)) answer.push_back(p);
91     }
92     if (answer.size() > 1 && same(answer.front(), answer.back())) answer.pop_back();
93     if (answer.size() < 3) return {};
94     return answer;
95 }
96 };

```

## 第三章 圆与多边形面积

### 3.1 圆与简单多边形相交面积

`area(points, circle)` 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。`edge(a, b, r)` 返回圆心在原点时有向边 `a` 到 `b` 的面积贡献，`r` 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和， $O(n)$  时间、 $O(1)$  额外空间。依赖浮点几何的 `Point/Circle` 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。

```
1 struct CirclePolygon
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     // Signed contribution of edge a -> b; circle centered at the origin.
9     static R edge(Point a, Point b, R r)
10    {
11        if (r == 0) return 0;
12        Point v = b - a;
13        R length = G::norm(v);
14        if (length == 0) return 0;
15        Point u = v / length;
16        R h = G::cross(u, a);
17        Point cuts[4] = {a};
18        int count = 1;
19        if (fabs1(h) <= r)
20        {
21            R delta = sqrt1(max(R(0), (r - fabs1(h)) * (r + fabs1(h))));
22            R t = -G::dot(a, u);
23            Point foot = G::perp(u) * h;
24            for (R s : {-delta, delta})
25                if (t + s > 0 && t + s < length) cuts[count++] = foot + u * s;
26        }
27        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```

28     R answer = 0;
29     for (int i = 1; i < count; i++)
30     {
31         Point p = cuts[i - 1], q = cuts[i];
32         R cross = G::cross(p, q);
33         if (G::norm((p + q) / 2) <= r)
34             answer += cross / 2;
35         else
36             answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
37     }
38     return answer;
39 }
40
41 // Simple polygon, either orientation; duplicate consecutive vertices allowed.
42 static R area(const vector<Point> &p, Circle c)
43 {
44     assert(c.r >= 0);
45     if (p.size() < 3 || c.r == 0) return 0;
46     R answer = 0, error = 0;
47     for (int i = 0; i < (int)p.size(); i++)
48     {
49         R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
50         R y = value - error;
51         R sum = answer + y;
52         error = (sum - answer) - y;
53         answer = sum;
54     }
55     return fabsl(answer);
56 }
57 };

```

## 第四章 最小覆盖圆

### 4.1 随机增量最小覆盖圆

`solve(points,rng,eps)` 返回 `optional<Circle>`, `Circle={o,r}`; 空集返回 `nullopt`, 单点或重复同一点返回半径零。`rng` 是调用者提供的 `mt19937_64` 随机数引擎。期望  $O(n)$  时间, 最坏  $O(n^3)$ ,  $O(n)$  空间; 随机化不可省略。`eps` 默认 32 倍 long double 机器精度, 用于相对半径的包含判断。`diameter(a,b)` 求直径圆; `through(a,b,c)` 求外接圆, 共线时返回最远点对的直径圆。依赖浮点几何的 `Point/Circle` 与向量运算。输入及中间运算须有限且尺度适当; 接近共线时圆心可能不稳定, 无法恢复读入前的精度。

```
1 struct EnclosingCircle
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     static Circle diameter(Point a, Point b)
9     {
10         Point o = a + (b - a) / 2;
11         return {o, max(G::norm(o - a), G::norm(o - b))};
12     }
13
14     static Circle through(Point a, Point b, Point c)
15     {
16         Point u = b - a, v = c - a;
17         R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
18         if (scale == 0) return {a, 0};
19         u = u / scale;
20         v = v / scale;
21         R det = 2 * G::cross(u, v);
22         if (det == 0)
23         {
24             Circle best = diameter(a, b);
25             for (Circle q : {diameter(a, c), diameter(b, c)})
26                 if (q.r > best.r) best = q;
27             return best;
28         }
29         R x = G::dot(u, u), y = G::dot(v, v);
30         Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
31         Point o = a + offset * scale;
32         return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
33     }
34 }
```

随机增量主过程（接上页同一结构体）：

```
35     static optional<Circle>
36     solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
37     {
38         assert(eps >= 0 && eps < 1);
39         if (p.empty()) return nullopt;
40         shuffle(p.begin(), p.end(), rng);
41         Circle c{p[0], 0};
42         auto outside = [&](Point q)
43         {
44             return G::norm(q - c.o) > c.r * (1 + eps);
45         };
46         for (int i = 1; i < (int)p.size(); i++)
47             if (outside(p[i]))
48             {
49                 c = {p[i], 0};
50                 for (int j = 0; j < i; j++)
51                     if (outside(p[j]))
52                     {
53                         c = diameter(p[i], p[j]);
54                         for (int k = 0; k < j; k++)
55                             if (outside(p[k])) c = through(p[i], p[j], p[k]);
56                     }
57             }
58         return c;
59     }
60 };
```

# 第五章 圆的切线

## 5.1 两圆公切线与过点切线

`solve(a,b,eps)` 返回 `Result`，包含 `infinite` 和 `lines`；`infinite` 为真表示无穷多条，否则 `lines` 给出全部切线。每条 `Tangent` 保存切点 `a`、`b`，单位法向量 `normal`，以及 `side`。`a`、`b` 对应输入两圆，直线方程  $\text{dot}(\text{normal}, x-a)=0$ ；切点相同时仍有效。`side=1` 外公切线、`-1` 内公切线、`0` 两个不同点。半径零表示点，`from_point(p,c,eps)` 等价于第一圆取点，圆上切点读 `b`。输入有限，坐标及半径绝对值  $\leq 1e100$ ，半径非负。`eps` 默认  $1e-12$ ，用于相对圆心距的近相切判断；近相切可能合并为一条，浮点坐标仍有舍入误差。严格同心等半径时 `infinite=true`，包括同一点；同心不等半径返回空。 $O(1)$ ；依赖浮点几何的 `Point/Circle`、点运算及 `norm/perp`。

```
1 struct CircleTangents
2 {
3     using G = RealGeometry;
4     using R = G::R;
5     using Point = G::Point;
6     using Circle = G::Circle;
7
8     struct Tangent
9     {
10         Point a, b, normal;
11         int side; // 1 outer, -1 inner, 0 two distinct point circles.
12     };
13
14     struct Result
15     {
16         bool infinite = false;
17         vector<Tangent> lines;
18     };
19
20     // Line: dot(normal, x - a) = 0. Radius zero represents a point.
21     static Result solve(Circle a, Circle b, R eps = 1e-12L)
22     {
23         assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
24         Point v = b.o - a.o;
25         R d = G::norm(v);
26         if (d == 0) return {a.r == b.r, {}};
27         Point u = v / d;
28         if (a.r == 0 && b.r == 0) return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```

29     Result answer;
30     for (int side : {1, -1})
31     {
32         if (side == -1 && (a.r == 0 || b.r == 0)) continue;
33         R gap = a.r - side * b.r;
34         if (fabs1(gap) > d * (1 + eps)) continue;
35         bool touch = fabs1(gap) >= d * (1 - eps);
36         R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
37         R s = sqrt1(max(R(0), (1 - c) * (1 + c)));
38         for (int sign : {-1, 1})
39         {
40             Point normal = u * c + G::perp(u) * (sign * s);
41             Point p = a.o + normal * a.r;
42             Point q = b.o + normal * (side * b.r);
43             answer.lines.push_back({p, q, normal, side});
44             if (touch) break;
45         }
46     }
47     return answer;
48 }
49
50 static Result from_point(Point p, Circle c, R eps = 1e-12L)
51 {
52     return solve({p, 0}, c, eps);
53 }
54 };

```

## 第六章 浮点最近点对

### 6.1 浮点平面最近点对

`solve(points)` 返回 `long double` 最短距离 (不是平方), 少于两个点返回 `nullopt`, 重复点返回零。Point={x,y}; 输入须有限, 坐标绝对值  $\leq 1e100$ 。递归分治, 按纵坐标归并,  $O(n \log n)$  时间、 $O(n)$  额外空间。用 `hypotl` 求距离, 不先平方; 排序不使用 `epsilon`。浮点运算仍有舍入误差, 输入前已丢失的精度无法恢复。

```
1 struct ClosestPair
2 {
3     using R = long double;
4
5     struct Point
6     {
7         R x, y;
8     };
9
10    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
11    static optional<R> solve(vector<Point> p)
12    {
13        int n = p.size();
14        if (n < 2) return nullopt;
15        sort(p.begin(),
16             p.end(),
17             [](Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); });
18        for (int i = 1; i < n; i++)
19            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y) return R(0);
20        auto by_y = [](Point a, Point b)
21        {
22            return tie(a.y, a.x) < tie(b.y, b.x);
23        };
24        auto distance = [](Point a, Point b)
25        {
26            return hypotl(a.x - b.x, a.y - b.y);
27        };
28        vector<Point> tmp(n);
29        function<R(int, int)> dfs = [&](int l, int r) -> R
30        {
31            if (r - l <= 3)
32            {
33                R best = numeric_limits<R>::infinity();
34                for (int i = l; i < r; i++)
35                    for (int j = l; j < i; j++) best = min(best, distance(p[i], p[j]));
36                sort(p.begin() + l, p.begin() + r, by_y);
37                return best;
38            }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
39     int m = (l + r) / 2;
40     R x = p[m].x;
41     R best = min(dfs(l, m), dfs(m, r));
42     merge(p.begin() + l,
43           p.begin() + m,
44           p.begin() + m,
45           p.begin() + r,
46           tmp.begin() + l,
47           by_y);
48     copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
49     vector<Point> strip;
50     for (int i = l; i < r; i++)
51         if (fabs1(p[i].x - x) <= best)
52         {
53             for (int j = (int)strip.size() - 1; j >= 0; j--)
54             {
55                 if (p[i].y - strip[j].y > best) break;
56                 best = min(best, distance(p[i], strip[j]));
57             }
58             strip.push_back(p[i]);
59         }
60     return best;
61 };
62 return dfs(0, n);
63 }
64 };
```

## 第七章 精确几何进阶

### 7.1 整数最近点对与 Minkowski 和

`closest_pair(points)` 返回最短距离平方，少于两个点为 `nullopt`，重复点返回 0， $O(n \log n)$ ，全程精确整数。  
`minkowski` 输入 strict CCW 凸包，返回不重复首点的凸包；非退化  $O(n+m)$ ，退化时  $O((n+m)\log(n+m))$ 。坐标及坐标和须满足整数几何范围。

```
1 struct GeometryExtra
2 {
3     using G = IntegerGeometry;
4     using P = G::Point;
5     using I = G::I;
6
7     // No pair exists for n<2. Exact squared distance,  $O(n \log n)$ .
8     static optional<I> closest_pair(vector<P> p)
9     {
10         int n = p.size();
11         if (n < 2) return nullopt;
12         sort(p.begin(), p.end());
13         for (int i = 1; i < n; i++)
14             if (p[i] == p[i - 1]) return I(0);
15         vector<P> tmp(n);
16         auto by_y = [](P a, P b)
17         {
18             return tie(a.y, a.x) < tie(b.y, b.x);
19         };
20         function<I(int, int)> solve = [&](int l, int r) -> I
21         {
22             if (r - l <= 3)
23             {
24                 I best = I(1) << 120;
25                 for (int i = l; i < r; i++)
26                     for (int j = i + 1; j < r; j++)
27                         best = min(best, G::dist2(p[i], p[j]));
28                 sort(p.begin() + l, p.begin() + r, by_y);
29                 return best;
30             }
31             int m = (l + r) / 2;
32             long long x = p[m].x;
33             I best = min(solve(l, m), solve(m, r));
34             merge(p.begin() + l,
35                  p.begin() + m,
36                  p.begin() + m,
37                  p.begin() + r,
38                  tmp.begin() + l,
39                  by_y);
40             copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
41             vector<P> strip;
42             for (int i = l; i < r; i++)
43                 if ((I(p[i].x) - x) * (I(p[i].x) - x) < best)
44                     {
```

```

45         for (int j = (int)strip.size() - 1; j >= 0; j--)
46         {
47             I dy = I(p[i].y) - strip[j].y;
48             if (dy * dy >= best) break;
49             best = min(best, G::dist2(p[i], strip[j]));
50         }
51         strip.push_back(p[i]);
52     }
53     return best;
54 };
55 return solve(0, n);
56 }
57
58 // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
59 static vector<P> minkowski(vector<P> a, vector<P> b)
60 {
61     if (a.empty() || b.empty()) return {};
62     auto plus = [](P u, P v)
63     {
64         return P{u.x + v.x, u.y + v.y};
65     };
66     if (a.size() < 3 || b.size() < 3)
67     {
68         vector<P> p;
69         for (P u : a)
70             for (P v : b) p.push_back(plus(u, v));
71         return G::hull(p);
72     }
73     auto start = [](vector<P> &p)
74     {
75         auto it =
76             min_element(p.begin(),
77                         p.end(),
78                         [](P u, P v) { return tie(u.y, u.x) < tie(v.y, v.x); });
79         rotate(p.begin(), it, p.end());
80     };
81     start(a);
82     start(b);
83     int n = a.size(), m = b.size(), i = 0, j = 0;
84     vector<P> result;
85     while (i < n || j < m)
86     {
87         result.push_back(plus(a[i % n], b[j % m]));
88         if (i == n)
89         {
90             ++j;
91             continue;
92         }
93         if (j == m)
94         {
95             ++i;
96             continue;
97         }

```

```
98         P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
99         P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
100         I c = G::cross({0, 0}, u, v);
101         if (c >= 0) ++i;
102         if (c <= 0) ++j;
103     }
104     return result;
105 }
106 };
```

## 7.2 整数三维共线、共面与方向

坐标绝对值  $\leq 10^9$ ;  $\text{orient}(a,b,c,d)$  为有向六倍四面体体积, 0 表示共面;  $\text{collinear}$  判三点共线,  $\text{on\_segment}$  支持退化线段。int128 精确判定, 不包含三维凸包构造。

```

1 struct IntegerGeometry3D
2 {
3     using I = __int128_t;
4
5     // |coordinates|<=1e9: triple products fit signed int128.
6     struct Point
7     {
8         long long x, y, z;
9     };
10
11     struct Vector
12     {
13         I x, y, z;
14     };
15
16     static Vector diff(Point a, Point b)
17     {
18         return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
19     }
20
21     static Vector cross(Vector a, Vector b)
22     {
23         return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
24     }
25
26     static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
27
28     static I orient(Point a, Point b, Point c, Point d)
29     {
30         return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
31     }
32
33     static bool collinear(Point a, Point b, Point c)
34     {
35         auto v = cross(diff(b, a), diff(c, a));
36         return !v.x && !v.y && !v.z;
37     }
38
39     static bool on_segment(Point p, Point a, Point b)
40     {
41         return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
42     }
43 };

```

# 第八章 凸包支撑点查询

## 8.1 支撑点查询恢复下凸包

静态 `build(left,right,oracle)` 返回从左到右的严格下凸包顶点, `Point` 为  $\{x,y\}$ 。有限非空整数点集无需显式给出; 回调 `oracle(a,b)` 必须精确返回使  $a*x+b*y$  最大的可行点, 允许任意最优解。 `left/right` 必须分别为最小/最大横坐标中纵坐标最低的点; 全体同横坐标时二者必须相同。所有坐标绝对值不超过  $10^{18}$ , 方向差使用 `long long`, 叉积先提升至 `int128`; 回调内部也须保证数值精确且无溢出。对弦  $a \rightarrow b$  查询方向  $(b.y-a.y, a.x-b.x)$ , 严格位于弦下方则递归分治, 否则确认边; 输出时删除共线中间点。若分治找到  $K$  个严格改善支撑点, 查询次数为  $2K+1$  (同横坐标时为 0), 额外时间、空间和最坏递归深度为  $O(K+1)$ 。  $K$  包括可能返回的共线非顶点, 不将其等同于最终顶点数。不能直接传浮点近似优化器。二分图闭包组合: 点为  $(|S|, |N(S)|)$ , 左点赋权  $a$ 、右点赋权  $b$ , 左  $\rightarrow$  邻点依赖; 下凸包方向满足  $a \geq 0, b < 0$ , 最优闭包的右点恰为邻集。端点为  $(0,0)$  和  $(\text{左点数}, \text{全部左点的邻点数})$ , 有孤立右点时后者不等于全部右点数。该组合已做子集枚举对拍, 尚不代表关联赛题整题通过。

```
1 struct SupportHull
2 {
3     using ll = long long;
4     using I = __int128_t;
5
6     struct Point
7     {
8         ll x, y;
9     };
10
11     static I cross(Point a, Point b, Point c)
12     {
13         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
14     }
```

```

16 // oracle(a,b) maximizes a*x+b*y exactly; |coordinates| <= 1e18.
17 // left/right: lowest points at the minimum/maximum x.
18 template <class Oracle>
19 static vector<Point> build(Point left, Point right, Oracle &&oracle)
20 {
21     assert(left.x <= right.x);
22     vector<Point> hull{left};
23     auto dfs = [&](auto &&self, Point a, Point b) -> void
24     {
25         Point c = oracle(b.y - a.y, a.x - b.x);
26         I turn = cross(a, b, c);
27         assert(turn <= 0);
28         if (turn < 0)
29         {
30             assert(a.x < c.x && c.x < b.x);
31             self(self, a, c);
32             self(self, c, b);
33         }
34         else
35         {
36             while (hull.size() >= 2 &&
37                 cross(hull[hull.size() - 2], hull.back(), b) == 0)
38                 hull.pop_back();
39             hull.push_back(b);
40         }
41     };
42     if (left.x < right.x)
43         dfs(dfs, left, right);
44     else
45         assert(left.y == right.y);
46     return hull;
47 }
48 };

```

## 第九章 配套知识与验证范围

### 9.1 数值算法与几何

排序必须使用严格弱序；不能以“差小于  $\text{eps}$  则相等”定义通用排序比较器。叉积与行列式容易发生抵消；整数题优先精确整数判定，浮点构造单独处理误差。对于简单格点多边形（无孔），Pick 定理  $A = I + B/2 - 1$ ，边界格点数  $B = \sum \gcd(|\Delta x|, |\Delta y|)$ ；不要重复计入边端点。自适应积分的局部误差估计需要函数足够规则；奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性，整数域最后枚举剩余区间；二分答案要求判定单调性。

**来源索引** 本章为重新组织的公式速查，扩展范围依据 <https://oi-wiki.org/math/>。小模数 exLucas 与有限域拉格朗日插值已有双风格实现和本地验证；素数模平方根已有 Cipolla 实现；仍需补齐二次剩余的其他方法与定理、Min25、Newton 插值、子集卷积、行列式进阶、数值积分等内容及验证。本章公式不能代替这些待完成代码。

# 第十章 计算几何：精确判定与适用范围

## 10.1 整数系数的有界半平面交

实现见第 7 页。输入是不等式  $ax + by \leq c$ ，三个系数均为绝对值不超过  $10^9$  的整数。本接口要求交集有界，只输出正面积多边形；无解、点交集和线段交集都返回空向量。它不是一般的可行性分类接口，不可将其空返回直接解读为闭半平面交集必为空。无界输入不在约定范围内，也不偷偷添加一个人工边框。

两条非平行边界交点用齐次坐标  $(X, Y, D)$  表示：

$$D = a_1b_2 - b_1a_2, \quad X = c_1b_2 - b_1c_2, \quad Y = a_1c_2 - c_1a_2.$$

若  $D < 0$ ，三者同时取负，使  $D > 0$ 。测试  $aX + bY \leq cD$  即可判断交点位于半平面内，无需除法。系数界保证  $|X|, |Y|, D \leq 2 \times 10^{18}$ ，可用 64 位保存；相等判断的交叉乘积至多为  $4 \times 10^{36}$ ，使用 128 位。因此本接口内的构造、排序和位置判定都是精确整数运算。

按法向量的半圆与叉积排序，比较器不使用 epsilon。同方向平行约束按正规化后的常数项保留更紧者，避免除以法向量长度。扫描时若新约束排除队首或队尾交点，就删除相应无效边界；每条边界最多入队、出队一次，排序后为线性时间。最后闭合队列，清除重复顶点；有界的非正面积结果不会输出多边形。零法向量表示常真约束或直接矛盾，先单独处理。

测试枚举所有边界对的交点，用任意精度整数逐条验证可行性，将可行顶点集合与返回集合比较，并检查循环顺序的严格凸性。两套版本通过 5000 组有界随机输入、平行和退化案例、近乎平行的十亿系数，以及十万条冗余约束的规模测试；普通与 ASan/UBSan 均通过。将有理数坐标转换为浮点、计算面积或处理浮点输入属于另一个数值问题，尚未由此验证。WIDA 的一般半平面交条目保留部分完成状态，浮点版本与无界分类仍需补齐。

## 10.2 最近点对、Minkowski 和与三维整数判定

最近点对与 Minkowski 和见第 18 页。最近点对返回距离平方，不开方；少于两个点时返回空 optional，重复点返回零。分治使用递归，每层按纵坐标归并，总时间  $O(n \log n)$ ，额外空间  $O(n)$ 。坐标绝对值上限为  $10^{12}$ ，作差后平方使用 128 位。

Minkowski 和输入必须是严格逆时针凸包，不重复首点，不含边上的冗余共线点；允许循环移动起点，也允许空集、单点和线段。输入坐标以及任意两个输入点的坐标和都须在  $[-10^{12}, 10^{12}]$  内。非退化凸包按边方向归并，时间  $O(n + m)$ ；退化分支枚举点和后求凸包，时间  $O((n + m) \log(n + m))$ 。返回同样不重复首点的严格凸包。

两套代码通过任意精度整数距离枚举，以及对所有点和独立执行礼品包装法的对拍；覆盖退化、循环起点和大坐标。规模测试包含二十万点最近点对，以及 40001 顶点的抛物线凸包与自身求和，结果应为原凸包各坐标的两倍。这些属于本地验证，不表示新增在线 AC。

三维判定见第 21 页。点的三个坐标为  $x, y, z$ ，绝对值均不超过  $10^9$ 。`orient(a, b, c, d)` 返回有向六倍四面体体积，零表示四点共面，交换任意两个点会改变符号。`collinear` 判断三点共线，`on_segment` 包含端点，且支持零长度线段。传统版线段接口名为 `On_Segment`。

点差每个分量至多  $2 \times 10^9$ ，叉积分量至多  $8 \times 10^{18}$ ，三重积绝对值至多  $4.8 \times 10^{28}$ ，在有符号 128 位范围内。公开的叉积、点积辅助函数不对任意 128 位向量保证不溢出；上述保证针对点差及其叉积。任意精度整数按  $4 \times 4$  行列式的 24 个排列独立计算方向，并用二阶子式及坐标区间验证共线与在线段上。已通过立方体八个顶点的全部有序四元组、随机十亿级坐标、换序及退化测试，普通与 ASan/UBSan 均通过。这里不包含三维凸包构造。

## 10.3 浮点最近点对

实现见第 16 页。`ClosestPair::solve` 与传统版 `Closest_Pair::Solve` 接收 `Point{x,y}` 数组，返回最短距离，不是距离平方。少于两点返回空 optional；重复点返回零。输入为有限 `long double`，坐标绝对值不超过  $10^{100}$ 。

递归前按横坐标排序，每次在递归改变数组前保存分界横坐标。左右区间分别递归后按纵坐标归并，再检查分界线附近的点对。时间  $O(n \log n)$ ，额外空间  $O(n)$ 。直接使用 `hypotl`，避免先平方造成上溢或下溢。排序比较器不使用 `epsilon`；不能把相近坐标随意视为相同点。

浮点结果仍有舍入误差，也无法恢复读入之前已经丢失的精度。整数坐标要求精确判定时，优先使用整数最近点对，最后按题目要求开方输出。一百位十进制浮点枚举所有点对，验证两套实现，覆盖小数、相邻可表示浮点数、极小非正规数、 $10^{-100}$  至  $10^{100}$  的尺度，以及二十万点规模；普通与 ASan/UBSan 均通过。本地完整 P1429 驱动测试包含 103 组输入和四位小数输出检查。在线评测的整数输入范围不能替代这里对小数与极端尺度的检查。

两套实现已在 P1429 通过全部 11 个测试点：[动态版记录 297602650](#)、[传统版记录 297603238](#)。单点最大用时均为 75 ms，内存峰值均为 12.32 MB。

## 10.4 公切线与过点切线

实现见第 14 页，依赖浮点几何的点、圆和向量运算。令两圆心差为  $v$ ， $d = |v|$ ， $u = v/d$ 。外公切线取  $s = 1$ ，内公切线取  $s = -1$ ，单位法向量满足

$$n \cdot u = \frac{r_1 - sr_2}{d} = c, \quad n = cu \pm \sqrt{(1-c)(1+c)} u^\perp.$$

切点为  $p = o_1 + r_1 n$ 、 $q = o_2 + sr_2 n$ 。直线以  $n \cdot (x - p) = 0$  表示，避免在两圆相切、两个切点相同时丢失方向。结果中的 **side** 为 1 或 -1，表示外公切线或内公切线。每个非相切分支有两条线，相切分支只输出一条。

半径零解释为点。有一个点圆时，只枚举外公切线分支，避免内外分支重复；两个不同点直接返回连线，**side**=0。严格同心且等半径返回 **infinite=true**，包括重合的两个点圆；严格同心但半径不同没有公切线。**from\_point(p,c)** 把第一圆设为点圆，圆上的切点读返回的 **b**。点在圆上时返回一条，点在圆内时空；点与零半径圆重合时返回无限条过点直线。

输入须有限，坐标和半径绝对值不超过  $10^{100}$ ，半径非负。容差  $0 < \varepsilon < 1$  默认  $10^{-12}$ ：当  $|r_1 - sr_2|$  与  $d$  的相对差在容差内时，合并为相切。因此极接近相切的输入可能返回一条近似切线，不承诺恢复舍入前的几何分类；使用方程时宜计算  $n \cdot (x - p)$ ，避免先计算大常数项造成额外消减。先比较半径差和圆心距，再做除法，避免极小圆心距造成比值溢出。

计数对拍使用任意精度整数比较圆心距平方与半径和、差的平方；一百位十进制运算验证切点到圆心的距离、圆心到切线的距离、法向量和切线一致性。两套实现通过 3000 组随机圆，在三个尺度下交换顺序测试，并覆盖重合、内外相切、点圆、极近同心与容差边界。普通与 ASan/UBSan 均通过；尚无此接口的在线 AC 记录。

## 10.5 随机增量最小覆盖圆

实现见第 12 页。随机打乱点集后依次加入。新点在当前圆外时，它必须成为新覆盖圆的边界点；以该点为固定边界重扫前缀，再遇到圆外点就固定两个边界点，最后由第三个圆外点确定外接圆。随机化使期望时间为  $O(n)$ ，最坏时间仍为  $O(n^3)$ 。传入初始化好的 `mt19937_64`，不可省略打乱。本实现使用三层循环，没有递归栈需求。

空集返回空 optional，单点及重复同一点的半径为零。直径圆用两端点到计算圆心的较大距离作为半径。三点外接圆先把第一点平移到原点，再按最大坐标差缩放后解两条垂直平分线方程；共线时使用最远点对的直径圆。**through** 对非共线三点求外接圆，不是任意三点的最小覆盖圆；例如钝角三角形的最小覆盖圆可能仅由最长边支撑，须调用主过程。

包含判断使用相对半径容差，默认 32 倍机器精度。输入与中间运算须有限、尺度适当；近共线三点的外接圆仍可能数值敏感，浮点比较不能恢复读入之前的精度。[P1742](#) 的驱动已准备，输出半径与圆心，该题坐标绝对值不超过  $10^4$ ，最多两位小数。

本地参考答案枚举所有单点、点对直径圆和三点外接圆，用一百位十进制运算筛出覆盖全部点且半径最小者，检查圆心、半径及覆盖性。两套版本通过 700 组随机小数点集，每组使用 12 个随机种子，另测共线、近共线、钝角/锐角三角形、平移后的小三角形、十万点直线与重复点。普通与 ASan/UBSan 均通过。两套代码已在 P1742 通过全部 10 个测试点：[动态版记录 297608403](#)、[传统版记录 297608801](#)。单点最大用时分别为 52 ms、49 ms，内存峰值为 6.50 MB、6.49 MB。该评测验证半径与圆心，不覆盖一并展开但未调用的其他几何接口。随机增量算法可参考 [Project Nayuki 的算法说明](#)。

## 10.6 圆与简单多边形相交面积

实现见第 10 页。先把圆心移到原点，再沿多边形有向边累加贡献，最后取绝对值。边与圆的交点把边分成至多三段。若小段中点在圆内，贡献是  $\text{cross}(p, q)/2$ ；否则贡献是有向扇形面积

$$\frac{r^2}{2} \text{atan2}(\text{cross}(p, q), p \cdot q).$$

必须保留符号，不能先对每条边取绝对值，否则凹多边形和圆外的边无法抵消。使用补偿求和减小大量边累加时的误差，总时间  $O(n)$ 、额外空间  $O(1)$ 。

求交点时令边的单位方向为  $u$ ，有向距离  $h = \text{cross}(u, a)$ ，垂足直接构造为  $hu^\perp$ ，交点为

$$hu^\perp \pm u\sqrt{(r - |h|)(r + |h|)}.$$

这避免用远端点加很长的位移来重建圆附近的交点。沿边的投影参数用于筛选位于线段内部的交点；端点本身已在分段列表内。相切交点重复不改变面积，零长度边返回零。

多边形须简单，支持凹形、两种方向和相邻重复顶点；半径非负，零半径或少于三个顶点时面积为零。输入与中间运算须有限且尺度适当，特别是平移后的乘积和面积要可表示。极小相交面积可能受到扇形贡献相消和近相切计算的影响，不作相对误差的统一保证。

测试包括完整圆、半圆、四分之一圆、内含多边形、相离、相切和高精度圆缺面积。独立参考实现沿竖直方向求多边形截线区间，再与圆的截线相交并数值积分，检查 150 组含凹形的随机多边形，另检查方向反转、重复点和平移。规模与数值测试含十万条边、极长的过圆心边和不同尺度。两套实现普通与 ASan/UBSan 均通过；尚无此模块的在线 AC。对应 kuangbin 二维几何中的圆与三角形、多边形相交面积部分。

# 算法索引

CirclePolygon, [10](#)  
CircleTangents, [14](#)  
ClosestPair, [16](#)  
  
EnclosingCircle, [12](#)  
  
GeometryExtra, [18](#)  
  
IntegerGeometry, [1](#)  
IntegerGeometry3D, [21](#)  
IntegerHalfplanes, [7](#)  
  
numerical stability, [24](#)  
  
Pick theorem, [24](#)  
  
RealGeometry, [3](#)  
  
SupportHull, [22](#)